Lesson 5: Enemy Ranks

Godot Engine

VIDEO GAME CREATION

This week: one enemy becomes many. Your job is to turn last week's single enemy into a whole row of them, using a loop instead of repeating yourself by hand.

Open your own [YourName]Game project from Lesson 4 - don't start a new one. Everything this week builds on the enemy and collision detection you already have.

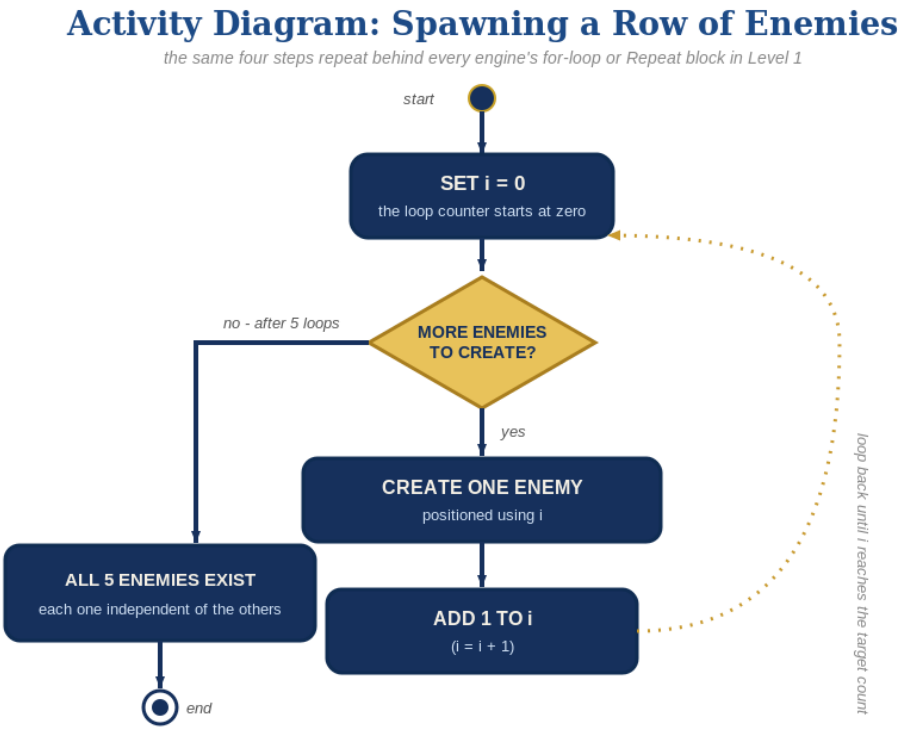
Tip: no new assets this week - you're reusing bullet_yellow.png, sfx_shoot.wav, enemy_invader_red.png, and sfx_hit.wav from your Lesson 3 and 4 Assets folders.

Setup

1. Open your [YourName]Game project from Lesson 4.
2. Delete the single Enemy instance you manually placed in your main scene last week - it's about to be replaced by several spawned copies.
3. If your main scene's root node doesn't already have a script attached, attach one now (extends Node2D, or whatever your root's type is).

Level 1 - Minimal (MVP)

Several enemies sit on screen in a row, created by a loop instead of placed or spawned one at a time. Shooting any one of them destroys just that one. This is a complete finish for today - reaching this is enough to move on to next week.



Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.

The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.

4. In the main scene's script, preload enemy.tscn near the top, then in _ready(), use a for loop to instantiate several copies, each positioned at a different x (code below).
5. Run the scene - you should see a row of enemies, and shooting any one of them should destroy just that one.

```
# Main scene's script:
var enemy_scene := preload("res://enemy.tscn")

func _ready():
    for enemy_count in range(5):
        var new_enemy := enemy_scene.instantiate()
        new_enemy.position = Vector2(150 + enemy_count * 200, 50)
        add_child(new_enemy)
```

Level 2 - Complete (Alpha/Beta)

Every enemy in the row can be individually shot and destroyed without affecting the others, and a bullet that misses every enemy still behaves exactly like it did last week.

- Shoot several different enemies in a row, including some misses - each hit destroys exactly the one enemy that was actually touched, and every other enemy is unaffected.
- This already works with no extra code: enemy.tscn's own script (its area_entered connection and hit-response) is self-contained, so every instance the loop creates behaves independently and correctly, exactly the way Lesson 4 built it.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: this already works with no changes - each spawned enemy carries its own Hit AudioStreamPlayer2D (if you built the Lesson 4 sound idea), so every instance plays it independently.
- Visual: vary each enemy's modulate color based on its loop index (enemy_count) at spawn time, for a striped row effect - alternate Color(0.8, 0.1, 0.2), a deep crimson red, with Color(0.95, 0.4, 0.5), a lighter dusty rose/pink (Godot's Color values run 0-1 instead of 0-255, so 0.8 is roughly 200 out of 255). The result is a row where every other enemy is a visibly different shade of red, not a smooth gradient blend.
- Structure: keep the spawn loop's only job being creating and positioning enemies - resist the urge to also handle collision-response logic in the main scene's script; that still belongs entirely inside enemy.tscn's own script.

If Something's Wrong

- Only one enemy appears, or they're all stacked in the same spot: confirm the loop variable enemy_count is actually used in the position calculation, not a fixed number.
- Nothing appears at all: confirm add_child(new_enemy) is actually being called for each instance - instantiate() alone creates the node in memory but doesn't add it to the scene tree.
- Old and new enemies both appear: confirm the manually-placed Lesson 4 Enemy instance was actually deleted from the scene, not just hidden.

Want More?

Student Resource List > Godot section, or docs.godotengine.org